

OOP_3

Sugeneruota Doxygen 1.9.8

Chapter 1

Hierarchijos Indeksas

1.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Vector< T >	??
Vector< int >	??
Zmogus	??
Studentas	??

Chapter 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	??
Vector< T >	??
Zmogus	??

Chapter 3

Failo Indeksas

3.1 Failai

Visų failų sąrašas su trumpais aprašymais:

include/ my_lib.h	??
include/ Studentas.h	??
include/ Vector.h	??
include/ Zmogus.h	??
src/ main.cpp	??
src/ my_lib.cpp	??
src/ vector_benchmark.cpp	??

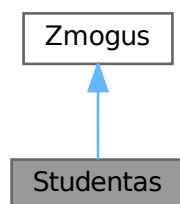
Chapter 4

Klasės Dokumentacija

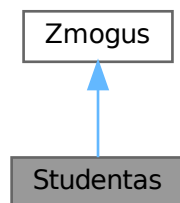
4.1 Studentas Klasė

```
#include <Studentas.h>
```

Paveldimumo diagrama Studentas:



Bendradarbiavimo diagrama Studentas:



Vieši Metodai

- `Studentas ()`
- `Studentas (const std::string &vardas, const std::string &pavarde, int egz, const Vector< int > &nd)`
- `Studentas (const Studentas &s)`
- `Studentas (Studentas &&s)`
- `~Studentas ()` override
- `Studentas & operator= (const Studentas &s)`
- `Studentas & operator= (Studentas &&s)`
- `std::string tipas ()` const override
- `int getEgz ()` const
- `const Vector< int > & getNd ()` const
- `double getGalVid ()` const
- `double getGalMed ()` const
- `void setEgz (int e)`
- `void setGalVid (double v)`
- `void setGalMed (double v)`
- `void pridetiNd (int pazymys)`
- `void isvalytiNd ()`
- `void pasalintiPaskutiniNd ()`
- `double skaiciuotiGalutiniSuVid ()` const
- `double skaiciuotiGalutiniSuMed ()` const

Vieši Metodai inherited from [Zmogus](#)

- `Zmogus ()`
- `Zmogus (std::string vardas, std::string pavarde)`
- `virtual ~Zmogus ()`
- `const std::string & getVardas ()` const
- `const std::string & getPavarde ()` const
- `void setVardas (const std::string &v)`
- `void setPavarde (const std::string &p)`

Draugai

- `std::ostream & operator<< (std::ostream &out, const Studentas &s)`
- `std::istream & operator>> (std::istream &in, Studentas &s)`

Additional Inherited Members

Apsaugoti Atributai inherited from [Zmogus](#)

- `std::string vardas\_`
- `std::string pavarde\_`

4.1.1 Konstruktorius ir Destruktorius Dokumentacija

4.1.1.1 `Studentas()` [1/4]

```
Studentas::Studentas ( ) [inline]
```

4.1.1.2 Studentas() [2/4]

```
Studentas::Studentas (
    const std::string & vardas,
    const std::string & pavarde,
    int egz,
    const Vector< int > & nd ) [inline]
```

4.1.1.3 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & s ) [inline]
```

4.1.1.4 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && s ) [inline]
```

4.1.1.5 ~Studentas()

```
Studentas::~~Studentas ( ) [inline], [override]
```

4.1.2 Metodų Dokumentacija**4.1.2.1 getEgz()**

```
int Studentas::getEgz ( ) const [inline]
```

4.1.2.2 getGalMed()

```
double Studentas::getGalMed ( ) const [inline]
```

4.1.2.3 getGalVid()

```
double Studentas::getGalVid ( ) const [inline]
```

4.1.2.4 getNd()

```
const Vector< int > & Studentas::getNd ( ) const [inline]
```

4.1.2.5 isvalytiNd()

```
void Studentas::isvalytiNd ( ) [inline]
```

4.1.2.6 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & s ) [inline]
```

4.1.2.7 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && s ) [inline]
```

4.1.2.8 pasalintiPaskutiniNd()

```
void Studentas::pasalintiPaskutiniNd ( ) [inline]
```

4.1.2.9 pridetiNd()

```
void Studentas::pridetiNd (
    int pazymys ) [inline]
```

4.1.2.10 setEgz()

```
void Studentas::setEgz (
    int e ) [inline]
```

4.1.2.11 setGalMed()

```
void Studentas::setGalMed (
    double v ) [inline]
```

4.1.2.12 setGalVid()

```
void Studentas::setGalVid (
    double v ) [inline]
```

4.1.2.13 skaiciuotiGalutiniSuMed()

```
double Studentas::skaiciuotiGalutiniSuMed ( ) const
```

4.1.2.14 skaiciuotiGalutiniSuVid()

```
double Studentas::skaiciuotiGalutiniSuVid ( ) const
```

4.1.2.15 tipas()

```
std::string Studentas::tipas ( ) const [inline], [override], [virtual]
```

Realizuoja [Zmogus](#).

4.1.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.1.3.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & s ) [friend]
```

4.1.3.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Studentas & s ) [friend]
```

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [include/Studentas.h](#)
- [src/my_lib.cpp](#)

4.2 Vector< T > Klasė Šablonas

```
#include <Vector.h>
```

Vieši Tipai

- using [value_type](#) = T
- using [size_type](#) = std::size_t
- using [difference_type](#) = std::ptrdiff_t
- using [reference](#) = [value_type](#) &
- using [const_reference](#) = const [value_type](#) &
- using [pointer](#) = [value_type](#) *
- using [const_pointer](#) = const [value_type](#) *
- using [iterator](#) = [value_type](#) *
- using [const_iterator](#) = const [value_type](#) *

Vieši Metodai

- `Vector ()`
- `Vector (size_type count)`
- `Vector (size_type count, const_reference value)`
- `Vector (std::initializer_list< value_type > values)`
- `Vector (const Vector &other)`
- `Vector & operator= (const Vector &other)`
- `Vector (Vector &&other) noexcept`
- `Vector & operator= (Vector &&other) noexcept`
- `~Vector ()`
- `reference operator[] (size_type index)`
- `const_reference operator[] (size_type index) const`
- `reference at (size_type index)`
- `const_reference at (size_type index) const`
- `reference front ()`
- `const_reference front () const`
- `reference back ()`
- `const_reference back () const`
- `pointer data ()`
- `const_pointer data () const`
- `iterator begin ()`
- `const_iterator begin () const`
- `iterator end ()`
- `const_iterator end () const`
- `size_type size () const`
- `size_type capacity () const`
- `bool empty () const`
- `void reserve (size_type newCapacity)`
- `void shrink_to_fit ()`
- `void push_back (const_reference value)`
- `void push_back (value_type &&value)`
- `void pop_back ()`
- `void clear ()`
- `void resize (size_type count)`
- `void resize (size_type count, const_reference value)`
- `void swap (Vector &other) noexcept`
- `iterator erase (const_iterator pos)`
- `iterator erase (const_iterator first, const_iterator last)`

4.2.1 Tipo Aprašymo Dokumentacija

4.2.1.1 const_iterator

```
template<typename T >
using Vector< T >::const_iterator = const value_type*
```

4.2.1.2 const_pointer

```
template<typename T >
using Vector< T >::const_pointer = const value_type*
```

4.2.1.3 const_reference

```
template<typename T >
using Vector< T >::const_reference = const value_type&
```

4.2.1.4 difference_type

```
template<typename T >
using Vector< T >::difference_type = std::ptrdiff_t
```

4.2.1.5 iterator

```
template<typename T >
using Vector< T >::iterator = value_type*
```

4.2.1.6 pointer

```
template<typename T >
using Vector< T >::pointer = value_type*
```

4.2.1.7 reference

```
template<typename T >
using Vector< T >::reference = value_type&
```

4.2.1.8 size_type

```
template<typename T >
using Vector< T >::size_type = std::size_t
```

4.2.1.9 value_type

```
template<typename T >
using Vector< T >::value_type = T
```

4.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.2.2.1 Vector() [1/6]

```
template<typename T >
Vector< T >::Vector ( ) [inline]
```

4.2.2.2 Vector() [2/6]

```
template<typename T >
Vector< T >::Vector (
    size_type count ) [inline], [explicit]
```

4.2.2.3 Vector() [3/6]

```
template<typename T >
Vector< T >::Vector (
    size_type count,
    const_reference value ) [inline]
```

4.2.2.4 Vector() [4/6]

```
template<typename T >
Vector< T >::Vector (
    std::initializer_list< value_type > values ) [inline]
```

4.2.2.5 Vector() [5/6]

```
template<typename T >
Vector< T >::Vector (
    const Vector< T > & other ) [inline]
```

4.2.2.6 Vector() [6/6]

```
template<typename T >
Vector< T >::Vector (
    Vector< T > && other ) [inline], [noexcept]
```

4.2.2.7 ~Vector()

```
template<typename T >
Vector< T >::~~Vector ( ) [inline]
```

4.2.3 Metodų Dokumentacija

4.2.3.1 at() [1/2]

```
template<typename T >
reference Vector< T >::at (
    size_type index ) [inline]
```


4.2.3.2 at() [2/2]

```
template<typename T >
const_reference Vector< T >::at (
    size_type index ) const [inline]
```

4.2.3.3 back() [1/2]

```
template<typename T >
reference Vector< T >::back ( ) [inline]
```

4.2.3.4 back() [2/2]

```
template<typename T >
const_reference Vector< T >::back ( ) const [inline]
```

4.2.3.5 begin() [1/2]

```
template<typename T >
iterator Vector< T >::begin ( ) [inline]
```

4.2.3.6 begin() [2/2]

```
template<typename T >
const_iterator Vector< T >::begin ( ) const [inline]
```

4.2.3.7 capacity()

```
template<typename T >
size_type Vector< T >::capacity ( ) const [inline]
```

4.2.3.8 clear()

```
template<typename T >
void Vector< T >::clear ( ) [inline]
```

4.2.3.9 data() [1/2]

```
template<typename T >
pointer Vector< T >::data ( ) [inline]
```

4.2.3.10 data() [2/2]

```
template<typename T >
const_pointer Vector< T >::data ( ) const [inline]
```

4.2.3.11 empty()

```
template<typename T >
bool Vector< T >::empty ( ) const [inline]
```

4.2.3.12 end() [1/2]

```
template<typename T >
iterator Vector< T >::end ( ) [inline]
```

4.2.3.13 end() [2/2]

```
template<typename T >
const_iterator Vector< T >::end ( ) const [inline]
```

4.2.3.14 erase() [1/2]

```
template<typename T >
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last ) [inline]
```

4.2.3.15 erase() [2/2]

```
template<typename T >
iterator Vector< T >::erase (
    const_iterator pos ) [inline]
```

4.2.3.16 front() [1/2]

```
template<typename T >
reference Vector< T >::front ( ) [inline]
```

4.2.3.17 front() [2/2]

```
template<typename T >
const_reference Vector< T >::front ( ) const [inline]
```

4.2.3.18 operator=() [1/2]

```
template<typename T >
Vector & Vector< T >::operator= (
    const Vector< T > & other ) [inline]
```

4.2.3.19 operator=() [2/2]

```
template<typename T >
Vector & Vector< T >::operator= (
    Vector< T > && other ) [inline], [noexcept]
```

4.2.3.20 operator[]() [1/2]

```
template<typename T >
reference Vector< T >::operator[] (
    size_type index ) [inline]
```

4.2.3.21 operator[]() [2/2]

```
template<typename T >
const_reference Vector< T >::operator[] (
    size_type index ) const [inline]
```

4.2.3.22 pop_back()

```
template<typename T >
void Vector< T >::pop_back ( ) [inline]
```

4.2.3.23 push_back() [1/2]

```
template<typename T >
void Vector< T >::push_back (
    const_reference value ) [inline]
```

4.2.3.24 push_back() [2/2]

```
template<typename T >
void Vector< T >::push_back (
    value_type && value ) [inline]
```

4.2.3.25 reserve()

```
template<typename T >
void Vector< T >::reserve (
    size_type newCapacity ) [inline]
```

4.2.3.26 resize() [1/2]

```
template<typename T >
void Vector< T >::resize (
    size_type count ) [inline]
```

4.2.3.27 `resize()` [2/2]

```
template<typename T >
void Vector< T >::resize (
    size_type count,
    const_reference value ) [inline]
```

4.2.3.28 `shrink_to_fit()`

```
template<typename T >
void Vector< T >::shrink_to_fit ( ) [inline]
```

4.2.3.29 `size()`

```
template<typename T >
size_type Vector< T >::size ( ) const [inline]
```

4.2.3.30 `swap()`

```
template<typename T >
void Vector< T >::swap (
    Vector< T > & other ) [inline], [noexcept]
```

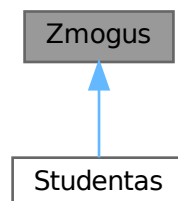
Dokumentacija šiai klasei sugeneruota iš šio failo:

- `include/Vector.h`

4.3 Zmogus Klasė

```
#include <Zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus](#) ()
- [Zmogus](#) (std::string vardas, std::string pavarde)
- virtual [~Zmogus](#) ()
- virtual std::string [tipas](#) () const =0
- const std::string & [getVardas](#) () const
- const std::string & [getPavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)

Apsaugoti Atributai

- std::string [vardas_](#)
- std::string [pavarde_](#)

4.3.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.3.1.1 Zmogus() [1/2]

```
Zmogus::Zmogus ( ) [inline]
```

4.3.1.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    std::string vardas,
    std::string pavarde ) [inline]
```

4.3.1.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus ( ) [inline], [virtual]
```

4.3.2 Metodų Dokumentacija

4.3.2.1 getPavarde()

```
const std::string & Zmogus::getPavarde ( ) const [inline]
```

4.3.2.2 getVardas()

```
const std::string & Zmogus::getVardas ( ) const [inline]
```

4.3.2.3 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p ) [inline]
```

4.3.2.4 setVardas()

```
void Zmogus::setVardas (
    const std::string & v ) [inline]
```

4.3.2.5 tipas()

```
virtual std::string Zmogus::tipas ( ) const [pure virtual]
```

Realizuota [Studentas](#).

4.3.3 Atributų Dokumentacija

4.3.3.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

4.3.3.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- include/[Zmogus.h](#)

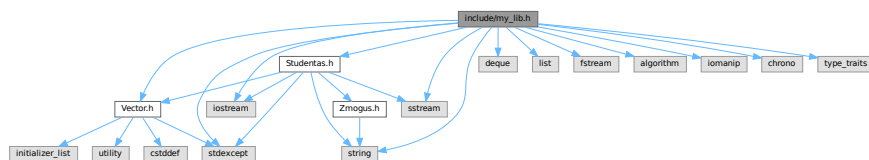
Chapter 5

Failo Dokumentacija

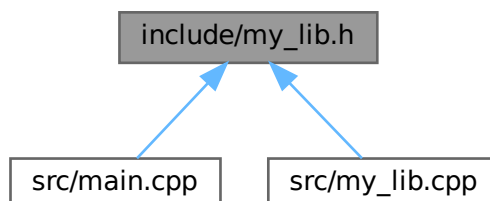
5.1 include/my_lib.h Failo Nuoroda

```
#include "Studentas.h"  
#include "Vector.h"  
#include <string>  
#include <deque>  
#include <list>  
#include <fstream>  
#include <sstream>  
#include <algorithm>  
#include <stdexcept>  
#include <iostream>  
#include <iomanip>  
#include <chrono>  
#include <type_traits>
```

Įtraukimo priklausomybių diagrama my_lib.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- void `rodytiRezultatus` (const `Vector< Studentas >` &studentai)
- void `generuotiPazymius` (`Studentas` &s)
- void `generuotiStudentus` (`Vector< Studentas >` &studentai)
- void `rodytiRez` (const `Vector< Studentas >` &studentai)
- bool `rikiuotiVarda` (const `Studentas` &A, const `Studentas` &B)
- bool `rikiuotiPavarde` (const `Studentas` &A, const `Studentas` &B)
- bool `rikiuotiPagalGalutiniVid` (const `Studentas` &A, const `Studentas` &B)
- bool `rikiuotiPagalGalutiniMed` (const `Studentas` &A, const `Studentas` &B)
- void `nuskaitytiFailaTestavimui` (`Vector< Studentas >` &studentai, int kartai)
- void `generuotiFaila` (int studentuSk)
- void `isvestiDuFailus` (const `Vector< Studentas >` &vargsai, const `Vector< Studentas >` &kieti)
- void `atliktiPirmaTyrima` ()
- void `atliktiAntraTyrima` ()
- template<typename Konteineris >
void `nuskaitytiFaila` (Konteineris &studentai, const std::string &failoVardas)
- template<typename Konteineris >
void `rikiuotiStudentus` (Konteineris &studentai, int pasirinkimas)
- template<typename Konteineris >
void `paskirstytiStudentus` (const Konteineris &studentai, Konteineris &vargsai, Konteineris &kieti)
- template<typename Konteineris >
void `paskirstytiStudentusS2` (Konteineris &studentai, Konteineris &vargsai)
- template<typename Konteineris >
void `paskirstytiStudentusS3` (Konteineris &studentai, Konteineris &vargsai)
- template<typename Konteineris >
void `atliktiAntraTyrima` (int strategija)

5.1.1 Funkcijos Dokumentacija

5.1.1.1 atliktiAntraTyrima() [1/2]

```
void atliktiAntraTyrima ( )
```

5.1.1.2 atliktiAntraTyrima() [2/2]

```
template<typename Konteineris >
void atliktiAntraTyrima (
    int strategija )
```

5.1.1.3 atliktiPirmaTyrima()

```
void atliktiPirmaTyrima ( )
```

5.1.1.4 generuotiFaila()

```
void generuotiFaila (
    int studentuSk )
```


5.1.1.5 generuotiPazymius()

```
void generuotiPazymius (
    Studentas & s )
```

5.1.1.6 generuotiStudentus()

```
void generuotiStudentus (
    Vector< Studentas > & studentai )
```

5.1.1.7 isvestiDuFailus()

```
void isvestiDuFailus (
    const Vector< Studentas > & vargsai,
    const Vector< Studentas > & kieti )
```

5.1.1.8 nuskaitytiFaila()

```
template<typename Konteineris >
void nuskaitytiFaila (
    Konteineris & studentai,
    const std::string & failoVardas )
```

5.1.1.9 nuskaitytiFailaTestavimui()

```
void nuskaitytiFailaTestavimui (
    Vector< Studentas > & studentai,
    int kartai )
```

5.1.1.10 paskirstytiStudentus()

```
template<typename Konteineris >
void paskirstytiStudentus (
    const Konteineris & studentai,
    Konteineris & vargsai,
    Konteineris & kieti )
```

5.1.1.11 paskirstytiStudentusS2()

```
template<typename Konteineris >
void paskirstytiStudentusS2 (
    Konteineris & studentai,
    Konteineris & vargsai )
```

5.1.1.12 paskirstytiStudentusS3()

```
template<typename Konteineris >
void paskirstytiStudentusS3 (
    Konteineris & studentai,
    Konteineris & vargsai )
```

5.1.1.13 rikiuotiPagalGalutiniMed()

```
bool rikiuotiPagalGalutiniMed (
    const Studentas & A,
    const Studentas & B )
```

5.1.1.14 rikiuotiPagalGalutiniVid()

```
bool rikiuotiPagalGalutiniVid (
    const Studentas & A,
    const Studentas & B )
```

5.1.1.15 rikiuotiPavarde()

```
bool rikiuotiPavarde (
    const Studentas & A,
    const Studentas & B )
```

5.1.1.16 rikiuotiStudentus()

```
template<typename Konteineris >
void rikiuotiStudentus (
    Konteineris & studentai,
    int pasirinkimas )
```

5.1.1.17 rikiuotiVarda()

```
bool rikiuotiVarda (
    const Studentas & A,
    const Studentas & B )
```

5.1.1.18 rodytiRez()

```
void rodytiRez (
    const Vector< Studentas > & studentai )
```

5.1.1.19 rodytiRezultatus()

```
void rodytiRezultatus (
    const Vector< Studentas > & studentai )
```

5.2 my_lib.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef MYLIB_H
00002 #define MYLIB_H
00003
00004 #include "Studentas.h"
00005 #include "Vector.h"
00006
00007 #include <string>
00008 #include <deque>
00009 #include <list>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <algorithm>
00013 #include <stdexcept>
00014 #include <iostream>
00015 #include <iomanip>
00016 #include <chrono>
00017 #include <type_traits>
00018
00019 void rodytiRezultatus(const Vector<Studentas>& studentai);
00020 void generuotiPazymius(Studentas& s);
00021 void generuotiStudentus(Vector<Studentas>& studentai);
00022 void rodytiRez(const Vector<Studentas>& studentai);
00023 bool rikiuotiVarda(const Studentas& A, const Studentas& B);
00024 bool rikiuotiPavarde(const Studentas& A, const Studentas& B);
00025 bool rikiuotiPagalGalutiniVid(const Studentas& A, const Studentas& B);
00026 bool rikiuotiPagalGalutiniMed(const Studentas& A, const Studentas& B);
00027 void nuskaitytiFailaTestavimui(Vector<Studentas>& studentai, int kartai);
00028 void generuotiFaila(int studentuSk);
00029 void isvestiDuFailus(const Vector<Studentas>& vargsai, const Vector<Studentas>& kieti);
00030 void atliktiPirmaTyrima();
00031 void atliktiAntraTyrima();
00032
00033 // templates
00034
00035 template <typename Konteineris>
00036 void nuskaitytiFaila(Konteineris& studentai, const std::string& failoVardas)
00037 {
00038     std::ifstream failas(failoVardas);
00039     if (!failas) {
00040         throw std::runtime_error("Nepavyko atidaryti failo " + failoVardas);
00041     }
00042
00043     std::stringstream buffer;
00044     buffer << failas.rdbuf();
00045
00046     std::string eilute;
00047     std::getline(buffer, eilute);
00048
00049     while (std::getline(buffer, eilute)) {
00050         std::stringstream ss(eilute);
00051
00052         Studentas s;
00053         std::string vardas, pavarde;
00054         if (!(ss >> vardas >> pavarde)) {
00055             throw std::runtime_error("Blogas formatas eiluteje " + eilute);
00056         }
00057         s.setVardas(vardas);
00058         s.setPavarde(pavarde);
00059
00060         int pazymys;
00061         while (ss >> pazymys) {
00062             s.pridetiNd(pazymys);
00063         }
00064
00065         if (s.getNd().empty()) {
00066             throw std::runtime_error("Truksta pazymiui eiluteje " + eilute);
00067         }
00068
00069         s.setEgz(s.getNd().back());
00070         s.pasalintiPaskutiniNd();
00071
00072         s.setGalVid(s.skaiciuotiGalutiniSuVid());
00073         s.setGalMed(s.skaiciuotiGalutiniSuMed());
00074
00075         studentai.push_back(s);
00076     }
00077 }
00078
00079 template <typename Konteineris>
00080 void rikiuotiStudentus(Konteineris& studentai, int pasirinkimas)
00081 {
00082     if (pasirinkimas < 1 || pasirinkimas > 4) {

```

```

00083         throw std::out_of_range("Pasirinkimas turi buti 1-4");
00084     }
00085
00086     if (pasirinkimas == 1) {
00087         // perziuri kompiliavimo metu, kad butu kompiliuojamas tik reikalingas rikiavimas, ziuredamas
00088         i template type
00089         if constexpr (std::is_same<Konteineris, std::list<Studentas> >::value)
00090             studentai.sort(rikiuotiVarda);
00091         else std::sort(studentai.begin(), studentai.end(), rikiuotiVarda);
00092     } else if (pasirinkimas == 2) {
00093         if constexpr (std::is_same<Konteineris, std::list<Studentas> >::value)
00094             studentai.sort(rikiuotiPavarde);
00095         else std::sort(studentai.begin(), studentai.end(), rikiuotiPavarde);
00096     } else if (pasirinkimas == 3) {
00097         if constexpr (std::is_same<Konteineris, std::list<Studentas> >::value)
00098             studentai.sort(rikiuotiPagalGalutiniVid);
00099         else std::sort(studentai.begin(), studentai.end(), rikiuotiPagalGalutiniVid);
00100     } else {
00101         if constexpr (std::is_same<Konteineris, std::list<Studentas> >::value)
00102             studentai.sort(rikiuotiPagalGalutiniMed);
00103         else std::sort(studentai.begin(), studentai.end(), rikiuotiPagalGalutiniMed);
00104     }
00105 }
00106
00107 // strategija 1
00108 template <typename Konteineris>
00109 void paskirstytiStudentus(const Konteineris& studentai, Konteineris& vargsai,
00110     Konteineris& kieti) {
00111     for (const auto& s : studentai) {
00112         if (s.getGalVid() >= 5.0) {
00113             kieti.push_back(s);
00114         } else {
00115             vargsai.push_back(s);
00116         }
00117     }
00118 }
00119
00120 // strategija 2
00121 template <typename Konteineris>
00122 void paskirstytiStudentusS2(Konteineris& studentai, Konteineris& vargsai)
00123 {
00124     vargsai.clear();
00125
00126     // rikiuoti mazejimo tvarka kad vargsai butu gale
00127     if constexpr (std::is_same_v<Konteineris, std::list<Studentas> >)
00128     {
00129         studentai.sort([](const Studentas& a, const Studentas& b) {
00130             return a.getGalVid() > b.getGalVid();
00131         });
00132     }
00133     else
00134     {
00135         std::sort(studentai.begin(), studentai.end(),
00136             [](const Studentas& a, const Studentas& b) {
00137                 return a.getGalVid() > b.getGalVid();
00138             });
00139     }
00140
00141     while (!studentai.empty() && studentai.back().getGalVid() < 5.0)
00142     {
00143         vargsai.push_back(studentai.back());
00144         studentai.pop_back();
00145     }
00146 }
00147
00148 // strategija 3
00149 template <typename Konteineris>
00150 void paskirstytiStudentusS3(Konteineris& studentai, Konteineris& vargsai)
00151 {
00152     vargsai.clear();
00153
00154     // std::partition pertvarko konteineri taip kad visi elementai kurie tenkina salyga butu
00155     // pradzioje, o kiti gale.
00156     // Grazina iterator i pirmo elemento po true grupes pradzia
00157     auto mid = std::partition(studentai.begin(), studentai.end(),
00158         [](const Studentas& s) { return s.getGalVid() < 5.0; });
00159
00160     for (auto it = studentai.begin(); it != mid; ++it) {
00161         vargsai.push_back(*it);
00162     }
00163
00164     studentai.erase(studentai.begin(), mid);
00165 }
00166
00167 }
00168
00169
00170
00171
00172
00173
00174
00175
00176
00177
00178
00179
00180
00181
00182
00183
00184
00185
00186
00187
00188
00189
00190
00191
00192
00193
00194
00195
00196
00197
00198
00199
00200
00201
00202
00203
00204
00205
00206
00207
00208
00209
00210
00211
00212
00213
00214
00215
00216
00217
00218
00219
00220
00221
00222
00223
00224
00225
00226
00227
00228
00229
00230
00231
00232
00233
00234
00235
00236
00237
00238
00239
00240
00241
00242
00243
00244
00245
00246
00247
00248
00249
00250
00251
00252
00253
00254
00255
00256
00257
00258
00259
00260
00261
00262
00263
00264
00265
00266
00267
00268
00269
00270
00271
00272
00273
00274
00275
00276
00277
00278
00279
00280
00281
00282
00283
00284
00285
00286
00287
00288
00289
00290
00291
00292
00293
00294
00295
00296
00297
00298
00299
00300
00301
00302
00303
00304
00305
00306
00307
00308
00309
00310
00311
00312
00313
00314
00315
00316
00317
00318
00319
00320
00321
00322
00323
00324
00325
00326
00327
00328
00329
00330
00331
00332
00333
00334
00335
00336
00337
00338
00339
00340
00341
00342
00343
00344
00345
00346
00347
00348
00349
00350
00351
00352
00353
00354
00355
00356
00357
00358
00359
00360
00361
00362
00363
00364
00365
00366
00367
00368
00369
00370
00371
00372
00373
00374
00375
00376
00377
00378
00379
00380
00381
00382
00383
00384
00385
00386
00387
00388
00389
00390
00391
00392
00393
00394
00395
00396
00397
00398
00399
00400
00401
00402
00403
00404
00405
00406
00407
00408
00409
00410
00411
00412
00413
00414
00415
00416
00417
00418
00419
00420
00421
00422
00423
00424
00425
00426
00427
00428
00429
00430
00431
00432
00433
00434
00435
00436
00437
00438
00439
00440
00441
00442
00443
00444
00445
00446
00447
00448
00449
00450
00451
00452
00453
00454
00455
00456
00457
00458
00459
00460
00461
00462
00463
00464
00465
00466
00467
00468
00469
00470
00471
00472
00473
00474
00475
00476
00477
00478
00479
00480
00481
00482
00483
00484
00485
00486
00487
00488
00489
00490
00491
00492
00493
00494
00495
00496
00497
00498
00499
00500
00501
00502
00503
00504
00505
00506
00507
00508
00509
00510
00511
00512
00513
00514
00515
00516
00517
00518
00519
00520
00521
00522
00523
00524
00525
00526
00527
00528
00529
00530
00531
00532
00533
00534
00535
00536
00537
00538
00539
00540
00541
00542
00543
00544
00545
00546
00547
00548
00549
00550
00551
00552
00553
00554
00555
00556
00557
00558
00559
00560
00561
00562
00563
00564
00565
00566
00567
00568
00569
00570
00571
00572
00573
00574
00575
00576
00577
00578
00579
00580
00581
00582
00583
00584
00585
00586
00587
00588
00589
00590
00591
00592
00593
00594
00595
00596
00597
00598
00599
00600
00601
00602
00603
00604
00605
00606
00607
00608
00609
00610
00611
00612
00613
00614
00615
00616
00617
00618
00619
00620
00621
00622
00623
00624
00625
00626
00627
00628
00629
00630
00631
00632
00633
00634
00635
00636
00637
00638
00639
00640
00641
00642
00643
00644
00645
00646
00647
00648
00649
00650
00651
00652
00653
00654
00655
00656
00657
00658
00659
00660
00661
00662
00663
00664
00665
00666
00667
00668
00669
00670
00671
00672
00673
00674
00675
00676
00677
00678
00679
00680
00681
00682
00683
00684
00685
00686
00687
00688
00689
00690
00691
00692
00693
00694
00695
00696
00697
00698
00699
00700
00701
00702
00703
00704
00705
00706
00707
00708
00709
00710
00711
00712
00713
00714
00715
00716
00717
00718
00719
00720
00721
00722
00723
00724
00725
00726
00727
00728
00729
00730
00731
00732
00733
00734
00735
00736
00737
00738
00739
00740
00741
00742
00743
00744
00745
00746
00747
00748
00749
00750
00751
00752
00753
00754
00755
00756
00757
00758
00759
00760
00761
00762
00763
00764
00765
00766
00767
00768
00769
00770
00771
00772
00773
00774
00775
00776
00777
00778
00779
00780
00781
00782
00783
00784
00785
00786
00787
00788
00789
00790
00791
00792
00793
00794
00795
00796
00797
00798
00799
00800
00801
00802
00803
00804
00805
00806
00807
00808
00809
00810
00811
00812
00813
00814
00815
00816
00817
00818
00819
00820
00821
00822
00823
00824
00825
00826
00827
00828
00829
00830
00831
00832
00833
00834
00835
00836
00837
00838
00839
00840
00841
00842
00843
00844
00845
00846
00847
00848
00849
00850
00851
00852
00853
00854
00855
00856
00857
00858
00859
00860
00861
00862
00863
00864
00865
00866
00867
00868
00869
00870
00871
00872
00873
00874
00875
00876
00877
00878
00879
00880
00881
00882
00883
00884
00885
00886
00887
00888
00889
00890
00891
00892
00893
00894
00895
00896
00897
00898
00899
00900
00901
00902
00903
00904
00905
00906
00907
00908
00909
00910
00911
00912
00913
00914
00915
00916
00917
00918
00919
00920
00921
00922
00923
00924
00925
00926
00927
00928
00929
00930
00931
00932
00933
00934
00935
00936
00937
00938
00939
00940
00941
00942
00943
00944
00945
00946
00947
00948
00949
00950
00951
00952
00953
00954
00955
00956
00957
00958
00959
00960
00961
00962
00963
00964
00965
00966
00967
00968
00969
00970
00971
00972
00973
00974
00975
00976
00977
00978
00979
00980
00981
00982
00983
00984
00985
00986
00987
00988
00989
00990
00991
00992
00993
00994
00995
00996
00997
00998
00999
01000

```

```

00164
00165 template <typename Konteineris>
00166 void atliktiAntraTyrima(int strategija) {
00167     using namespace std::chrono;
00168     Vector<int> kiekiai = {1000, 10000, 100000, 1000000, 10000000};
00169     int rikiavimas = 3;
00170
00171     std::cout << "-----\n";
00172     std::cout << std::left << std::setw(10) << "Irasai"
00173               << std::setw(12) << "Skaitymas"
00174               << std::setw(12) << "Rikiavimas"
00175               << std::setw(12) << "Skirstymas"
00176               << "Viso\n";
00177     std::cout << "-----\n";
00178
00179     for (int n : kiekiai) {
00180         Konteineris studentai;
00181         Konteineris vargsai;
00182         Konteineris kieti;
00183         std::string failoVardas = "generuotiStud" + std::to_string(n) + ".txt";
00184
00185         auto visoPradzia = high_resolution_clock::now();
00186
00187         // 1 skaitymas
00188         auto s1 = high_resolution_clock::now();
00189         try {
00190             nuskaitytiFaila(studentai, failoVardas);
00191         } catch (const std::exception& e) {
00192             std::cerr << "Klaida: Nepavyko rasti " << failoVardas << "\n";
00193             continue;
00194         }
00195         auto e1 = high_resolution_clock::now();
00196
00197         // 2 rikiavimas didejimo tvarka
00198         auto s2 = high_resolution_clock::now();
00199         rikiuotiStudentus(studentai, rikiavimas);
00200         auto e2 = high_resolution_clock::now();
00201
00202         // 3 skirstymas
00203         auto s3 = high_resolution_clock::now();
00204         if (strategija==1) paskirstytiStudentus(studentai, vargsai, kieti);
00205         else if (strategija==2) paskirstytiStudentusS2(studentai, vargsai);
00206         else if (strategija==3) paskirstytiStudentusS3(studentai, vargsai);
00207         auto e3 = high_resolution_clock::now();
00208
00209         auto visoPabaiga = high_resolution_clock::now();
00210
00211         double trukme1 = duration<double>(e1 - s1).count();
00212         double trukme2 = duration<double>(e2 - s2).count();
00213         double trukme3 = duration<double>(e3 - s3).count();
00214         double trukmeViso = duration<double>(visoPabaiga - visoPradzia).count();
00215
00216         std::cout << std::left << std::setw(10) << n
00217               << std::fixed << std::setprecision(4)
00218               << std::setw(12) << trukme1
00219               << std::setw(12) << trukme2
00220               << std::setw(12) << trukme3
00221               << trukmeViso << " s\n";
00222
00223         studentai.clear();
00224         vargsai.clear();
00225         kieti.clear();
00226     }
00227     std::cout << "-----\n";
00228 }
00229
00230
00231 #endif

```

5.3 include/Studentas.h Failo Nuoroda

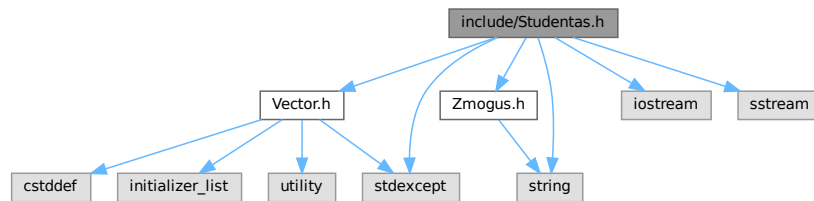
```

#include "Vector.h"
#include "Zmogus.h"
#include <stdexcept>
#include <string>
#include <iostream>

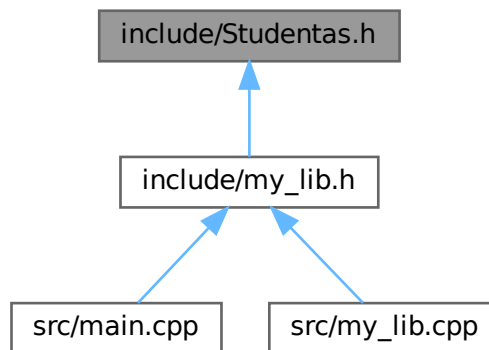
```

```
#include <sstream>
```

Įtraukimo priklausomybių diagrama Studentas.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [Studentas](#)

5.4 Studentas.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "Vector.h"
00005 #include "Zmogus.h"
00006
00007 #include <stdexcept>
00008 #include <string>
00009 #include <iostream>
00010 #include <sstream>
00011
00012 class Studentas : public Zmogus{
00013 private:
00014     int egz_;
00015     Vector<int> nd_;
  
```

```

00016     double galVid_;
00017     double galMed_;
00018
00019 public:
00020     Studentas() : Zmogus("", ""), egz_(0), nd_{{}}, galVid_(0.0), galMed_(0.0) {}
00021     Studentas(const std::string& vardas, const std::string& pavarde, int egz, const Vector<int>& nd)
00022         : Zmogus(vardas, pavarde), egz_(egz), nd_(nd), galVid_(0.0), galMed_(0.0) {}
00023
00024     // copy konstruktorius
00025     Studentas(const Studentas& s)
00026         : Zmogus(s.vardas_, s.pavarde_),
00027         egz_(s.egz_), nd_(s.nd_),
00028         galVid_(s.galVid_), galMed_(s.galMed_) {}
00029
00030     // move konstruktorius
00031     Studentas(Studentas&& s) // && - rvalue nuoroda, std::move pavercia i rvalue
00032         : Zmogus(std::move(s.vardas_), std::move(s.pavarde_)),
00033         egz_(s.egz_), nd_(std::move(s.nd_)),
00034         galVid_(s.galVid_), galMed_(s.galMed_) {
00035         s.egz_ = 0;
00036         s.galVid_ = 0.0;
00037         s.galMed_ = 0.0;
00038     }
00039
00040     // destruktorius
00041     ~Studentas() override {
00042         vardas_.clear();
00043         pavarde_.clear();
00044         nd_.clear();
00045         egz_ = 0;
00046         galVid_ = 0.0;
00047         galMed_ = 0.0;
00048     }
00049
00050     // copy assignment operatorius
00051     Studentas& operator=(const Studentas& s) {
00052         if (this == &s) return *this;
00053         vardas_ = s.vardas_;
00054         pavarde_ = s.pavarde_;
00055         egz_ = s.egz_;
00056         nd_ = s.nd_;
00057         galVid_ = s.galVid_;
00058         galMed_ = s.galMed_;
00059
00060         return *this;
00061     }
00062
00063     // move assignment operatorius
00064     Studentas& operator=(Studentas&& s) {
00065         if (this == &s) return *this;
00066         vardas_ = std::move(s.vardas_);
00067         pavarde_ = std::move(s.pavarde_);
00068         nd_ = std::move(s.nd_);
00069         egz_ = s.egz_;
00070         galVid_ = s.galVid_;
00071         galMed_ = s.galMed_;
00072
00073         s.egz_ = 0;
00074         s.galVid_ = 0.0;
00075         s.galMed_ = 0.0;
00076
00077         return *this;
00078     }
00079
00080     std::string tipas() const override { return "Studentas"; }
00081
00082     // output operatorius
00083     friend std::ostream& operator<<(std::ostream& out, const Studentas& s) {
00084         out << s.vardas_ << ' ' << s.pavarde_;
00085         for (int nd : s.nd_) out << ' ' << nd;
00086         out << ' ' << s.egz_;
00087         return out;
00088     }
00089
00090     // input operatorius
00091     friend std::istream& operator>>(std::istream& in, Studentas& s) {
00092         std::string eilute;
00093         std::getline(in, eilute);
00094         if (!in) return in;
00095
00096         std::stringstream ss(eilute);
00097
00098         if (!(ss >> s.vardas_ >> s.pavarde_)) {
00099             throw std::runtime_error("Blogas studento formatas: truksta vardo/pavardes");
00100         }
00101
00102         s.nd_.clear();

```

```

00103     int paz;
00104     while (ss » paz) s.nd_.push_back(paz);
00105
00106     if (s.nd_.empty()) {
00107         throw std::runtime_error("Blogas studento formatas: truksta pazymiu");
00108     }
00109
00110     s.egz_ = s.nd_.back();
00111     s.nd_.pop_back();
00112
00113     s.galVid_ = s.skaiciuotiGalutiniSuVid();
00114     s.galMed_ = s.skaiciuotiGalutiniSuMed();
00115     return in;
00116 }
00117
00118 int getEgz() const { return egz_; }
00119 const Vector<int>& getNd() const { return nd_; }
00120 double getGalVid() const { return galVid_; }
00121 double getGalMed() const { return galMed_; }
00122
00123 void setEgz(int e) { egz_ = e; }
00124 void setGalVid(double v) { galVid_ = v; }
00125 void setGalMed(double v) { galMed_ = v; }
00126
00127 void pridetiNd(int pazymys) { nd_.push_back(pazymys); }
00128 void isvalytiNd() { nd_.clear(); }
00129 void pasalintiPaskutiniNd() {
00130     if (nd_.empty()) throw std::runtime_error("Truksta pazymiu eiluteje");
00131     nd_.pop_back();
00132 }
00133
00134
00135 double skaiciuotiGalutiniSuVid() const;
00136 double skaiciuotiGalutiniSuMed() const;
00137 };
00138
00139 #endif

```

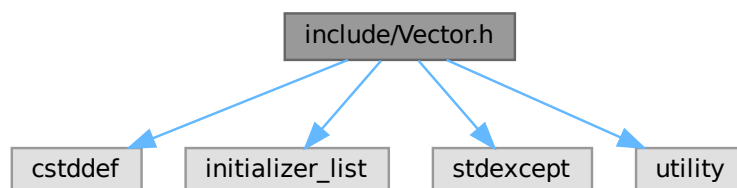
5.5 include/Vector.h Failo Nuoroda

```

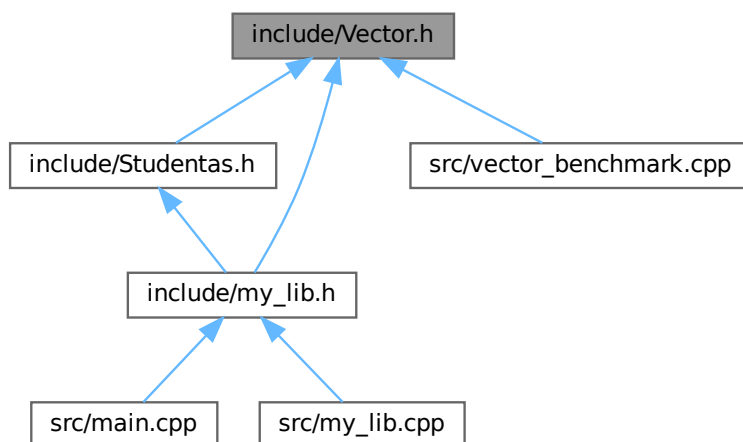
#include <cstdint>
#include <initializer_list>
#include <stdexcept>
#include <utility>

```

Įtraukimo priklausomybių diagrama Vector.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class `Vector< T >`

Funkcijos

- `template<typename T >`
`bool operator== (const Vector< T > &lhs, const Vector< T > &rhs)`
- `template<typename T >`
`bool operator!= (const Vector< T > &lhs, const Vector< T > &rhs)`

5.5.1 Funkcijos Dokumentacija

5.5.1.1 operator"!=()

```

template<typename T >
bool operator!= (
    const Vector< T > & lhs,
    const Vector< T > & rhs )
  
```

5.5.1.2 operator"==()

```

template<typename T >
bool operator== (
    const Vector< T > & lhs,
    const Vector< T > & rhs )
  
```

5.6 Vector.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <cstddef>
00005 #include <initializer_list>
00006 #include <stdexcept>
00007 #include <utility>
00008
00009 // pagal https://en.cppreference.com/cpp/container/vector
00010
00011 template <typename T>
00012 class Vector {
00013 public:
00014     //member types
00015     using value_type = T;
00016     using size_type = std::size_t;
00017     using difference_type = std::ptrdiff_t;
00018     using reference = value_type&;
00019     using const_reference = const value_type&;
00020     using pointer = value_type*;
00021     using const_pointer = const value_type*;
00022     using iterator = value_type*;
00023     using const_iterator = const value_type*;
00024
00025 private:
00026     pointer data_;
00027     size_type size_;
00028     size_type capacity_;
00029
00030     //helper funkcija capacity metodams, DRY :)
00031     void reallocate(size_type newCapacity)
00032     {
00033         pointer newData = newCapacity > 0 ? new value_type[newCapacity] : nullptr;
00034
00035         for (size_type i = 0; i < size_; i++) {
00036             newData[i] = data_[i];
00037         }
00038
00039         delete[] data_;
00040         data_ = newData;
00041         capacity_ = newCapacity;
00042     }
00043
00044     //helper funkcija skirta push_back
00045     void growIfFull()
00046     {
00047         if (size_ < capacity_) return;
00048
00049         reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00050     }
00051
00052 public:
00053     //member functions
00054     // default konstruktorius
00055     Vector() : data_(nullptr), size_(0), capacity_(0) {}
00056
00057     //konstruktorius su pradiniu size
00058     explicit Vector(size_type count)
00059         : data_(count > 0 ? new value_type[count]{} : nullptr),
00060         size_(count),
00061         capacity_(count) {}
00062
00063     //konstruktorius su pradiniu size ir reiksme
00064     Vector(size_type count, const_reference value)
00065         : data_(count > 0 ? new value_type[count] : nullptr),
00066         size_(count),
00067         capacity_(count)
00068     {
00069         for (size_type i = 0; i < size_; i++) {
00070             data_[i] = value;
00071         }
00072     }
00073
00074     //initializer list konstruktorius
00075     Vector(std::initializer_list<value_type> values)
00076         : data_(values.size() > 0 ? new value_type[values.size()] : nullptr),
00077         size_(values.size()),
00078         capacity_(values.size())
00079     {
00080         size_type i = 0;
00081         for (const auto& value : values) {
00082             data_[i] = value;

```

```

00083         i++;
00084     }
00085 }
00086
00087 //copy konstruktorius
00088 Vector(const Vector& other)
00089 : data_(other.size_ > 0 ? new value_type[other.size_] : nullptr),
00090   size_(other.size_),
00091   capacity_(other.size_)
00092 {
00093     for (size_type i = 0; i < size_; i++) {
00094         data_[i] = other.data_[i];
00095     }
00096 }
00097
00098 //copy assignment operatorius
00099 Vector& operator=(const Vector& other)
00100 {
00101     if (this == &other) return *this;
00102
00103     delete[] data_;
00104
00105     data_ = other.capacity_ > 0 ? new value_type[other.capacity_] : nullptr;
00106     size_ = other.size_;
00107     capacity_ = other.capacity_;
00108
00109     for (size_type i = 0; i < size_; i++) {
00110         data_[i] = other.data_[i];
00111     }
00112
00113     return *this;
00114 }
00115
00116 //move konstruktorius
00117 Vector(Vector&& other) noexcept
00118 : data_(other.data_),
00119   size_(other.size_),
00120   capacity_(other.capacity_)
00121 {
00122     other.data_ = nullptr;
00123     other.size_ = 0;
00124     other.capacity_ = 0;
00125 }
00126
00127 //move assignment operatorius
00128 Vector& operator=(Vector&& other) noexcept
00129 {
00130     if (this == &other) return *this;
00131
00132     delete[] data_;
00133     data_ = other.data_;
00134     size_ = other.size_;
00135     capacity_ = other.capacity_;
00136
00137     other.data_ = nullptr;
00138     other.size_ = 0;
00139     other.capacity_ = 0;
00140
00141     return *this;
00142 }
00143
00144 ~Vector()
00145 {
00146     delete[] data_;
00147 }
00148
00149 // element access
00150 reference operator[](size_type index)
00151 {
00152     return data_[index];
00153 }
00154
00155 const_reference operator[](size_type index) const
00156 {
00157     return data_[index];
00158 }
00159
00160 reference at(size_type index)
00161 {
00162     if (index >= size_) {
00163         throw std::out_of_range("Vektoriaus indeksas uz ribu");
00164     }
00165
00166     return data_[index];
00167 }
00168
00169 const_reference at(size_type index) const

```

```
00170     {
00171         if (index >= size_) {
00172             throw std::out_of_range("Vektoriaus indeksas uz ribu");
00173         }
00174
00175         return data_[index];
00176     }
00177
00178     reference front()
00179     {
00180         return data_[0];
00181     }
00182
00183     const_reference front() const
00184     {
00185         return data_[0];
00186     }
00187
00188     reference back()
00189     {
00190         return data_[size_ - 1];
00191     }
00192
00193     const_reference back() const
00194     {
00195         return data_[size_ - 1];
00196     }
00197
00198     pointer data()
00199     {
00200         return data_;
00201     }
00202
00203     const_pointer data() const
00204     {
00205         return data_;
00206     }
00207
00208     // iterators
00209     iterator begin()
00210     {
00211         return data_;
00212     }
00213
00214     const_iterator begin() const
00215     {
00216         return data_;
00217     }
00218
00219     iterator end()
00220     {
00221         return data_ + size_;
00222     }
00223
00224     const_iterator end() const
00225     {
00226         return data_ + size_;
00227     }
00228
00229     // capacity
00230     size_type size() const
00231     {
00232         return size_;
00233     }
00234
00235     size_type capacity() const
00236     {
00237         return capacity_;
00238     }
00239
00240     bool empty() const
00241     {
00242         return size_ == 0;
00243     }
00244
00245     void reserve(size_type newCapacity)
00246     {
00247         if (newCapacity <= capacity_) return;
00248
00249         reallocate(newCapacity);
00250     }
00251
00252     void shrink_to_fit()
00253     {
00254         if (capacity_ == size_) return;
00255
00256         reallocate(size_);
```

```

00257     }
00258
00259     // modifiers
00260     void push_back(const_reference value)
00261     {
00262         growIfFull();
00263         data_[size_] = value;
00264         size_++;
00265     }
00266
00267     void push_back(value_type&& value)
00268     {
00269         growIfFull();
00270         data_[size_] = std::move(value);
00271         size_++;
00272     }
00273
00274     void pop_back()
00275     {
00276         if (size_ > 0) {
00277             size_--;
00278         }
00279     }
00280     // docs: size() returns zero, leaves the capacity() of the vector unchanged
00281     void clear()
00282     {
00283         size_ = 0;
00284         // delete[] data;
00285         // data_ = nullptr;
00286         // capacity_ = 0;
00287     }
00288
00289     void resize(size_type count)
00290     {
00291         if (count > capacity_) {
00292             reserve(count);
00293         }
00294
00295         for (size_type i = size_; i < count; i++) {
00296             data_[i] = value_type{};
00297         }
00298
00299         size_ = count;
00300     }
00301
00302     void resize(size_type count, const_reference value)
00303     {
00304         if (count > capacity_) {
00305             reserve(count);
00306         }
00307
00308         for (size_type i = size_; i < count; i++) {
00309             data_[i] = value;
00310         }
00311
00312         size_ = count;
00313     }
00314
00315     void swap(Vector& other) noexcept
00316     {
00317         std::swap(data_, other.data_);
00318         std::swap(size_, other.size_);
00319         std::swap(capacity_, other.capacity_);
00320     }
00321
00322     iterator erase(const_iterator pos)
00323     {
00324         if (size_ == 0 || pos < data_ || pos >= data_ + size_) {
00325             throw std::out_of_range("Vektoriaus erase iteratorius uz ribu");
00326         }
00327
00328         return erase(pos, pos + 1);
00329     }
00330
00331     iterator erase(const_iterator first, const_iterator last)
00332     {
00333         if (first < data_ || last < first || last > data_ + size_) {
00334             throw std::out_of_range("Vektoriaus erase iteratorius uz ribu");
00335         }
00336
00337         size_type firstIndex = first - data_;
00338         size_type lastIndex = last - data_;
00339         size_type count = lastIndex - firstIndex;
00340         // perstumia likusius elementus i kaire
00341         for (size_type i = firstIndex; i + count < size_; i++) {
00342             data_[i] = std::move(data_[i + count]);
00343         }

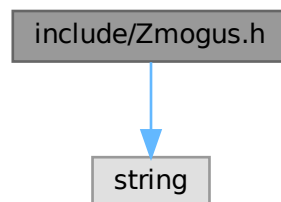
```

```
00344
00345     size_ -= count;
00346     return data_ + firstIndex;
00347 }
00348
00349
00350 };
00351
00352 // non-member functions
00353 // comparison operators
00354 template <typename T>
00355 bool operator==(const Vector<T>& lhs, const Vector<T>& rhs)
00356 {
00357     if (lhs.size() != rhs.size()) return false;
00358
00359     for (std::size_t i = 0; i < lhs.size(); i++) {
00360         if (lhs[i] != rhs[i]) return false;
00361     }
00362
00363     return true;
00364 }
00365
00366 template <typename T>
00367 bool operator!=(const Vector<T>& lhs, const Vector<T>& rhs)
00368 {
00369     return !(lhs == rhs);
00370 }
00371
00372 #endif
```

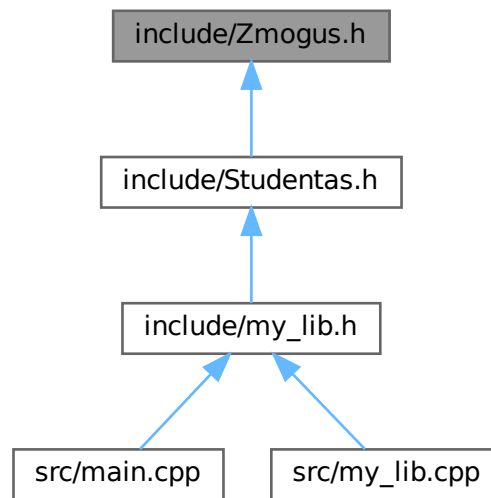
5.7 include/Zmogus.h Failo Nuoroda

```
#include <string>
```

Įtraukimo priklausomybių diagrama Zmogus.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [Zmogus](#)

5.8 Zmogus.h

Eiti į šio failo dokumentaciją.

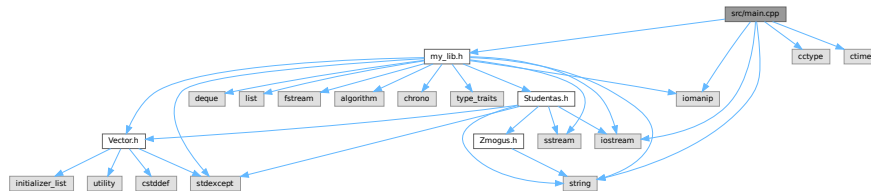
```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00006 class Zmogus {
00007 protected:
00008     std::string vardas_;
00009     std::string pavarde_;
00010
00011 public:
00012     Zmogus() : vardas_(""), pavarde_("") {}
00013
00014     Zmogus(std::string vardas, std::string pavarde) : vardas_(vardas), pavarde_(pavarde) {};
00015
00016     virtual ~Zmogus() {
00017         vardas_.clear();
00018         pavarde_.clear();
00019     };
00020
00021     // pure virtual funkcija paverčia klase abstrakcia
00022     virtual std::string tipas() const = 0;
00023
00024     const std::string& getVardas() const { return vardas_; }
00025     const std::string& getPavarde() const { return pavarde_; }
00026
00027     void setVardas(const std::string& v) { vardas_ = v; }
00028     void setPavarde(const std::string& p) { pavarde_ = p; }
00029 };
00030 #endif
  
```

5.9 src/main.cpp Failo Nuoroda

```
#include "my_lib.h"
#include <cctype>
#include <iomanip>
#include <iostream>
#include <string>
#include <ctime>
```

Įtraukimo priklausomybių diagrama main.cpp:



Funkcijos

- int [main](#) ()

5.9.1 Funkcijos Dokumentacija

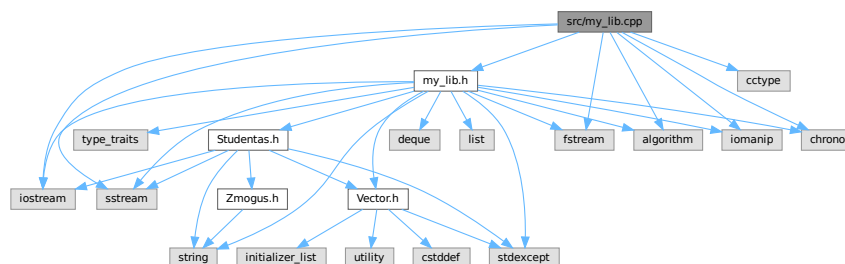
5.9.1.1 main()

```
int main ( )
```

5.10 src/my_lib.cpp Failo Nuoroda

```
#include "my_lib.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <cctype>
#include <chrono>
#include <sstream>
#include <fstream>
```

Įtraukimo priklausomybių diagrama my_lib.cpp:



Funkcijos

- void `rodytiRezultatus` (const `Vector< Studentas >` &studentai)
- void `generuotiPazymius` (`Studentas` &s)
- void `generuotiStudentus` (`Vector< Studentas >` &studentai)
- void `rodytiRez` (const `Vector< Studentas >` &studentai)
- bool `rikiuotiVarda` (const `Studentas` &A, const `Studentas` &B)
- bool `rikiuotiPavarde` (const `Studentas` &A, const `Studentas` &B)
- bool `rikiuotiPagalGalutiniVid` (const `Studentas` &A, const `Studentas` &B)
- bool `rikiuotiPagalGalutiniMed` (const `Studentas` &A, const `Studentas` &B)
- void `nuskaitytiFailaTestavimui` (`Vector< Studentas >` &studentai, int kartai)
- void `generuotiFaila` (int studentuSk)
- void `isvestiDuFailus` (const `Vector< Studentas >` &vargsai, const `Vector< Studentas >` &kiet)
- void `atliktiPirmaTyrima` ()

5.10.1 Funkcijos Dokumentacija

5.10.1.1 atliktiPirmaTyrima()

```
void atliktiPirmaTyrima ( )
```

5.10.1.2 generuotiFaila()

```
void generuotiFaila (
    int studentuSk )
```

5.10.1.3 generuotiPazymius()

```
void generuotiPazymius (
    Studentas & s )
```

5.10.1.4 generuotiStudentus()

```
void generuotiStudentus (
    Vector< Studentas > & studentai )
```

5.10.1.5 isvestiDuFailus()

```
void isvestiDuFailus (
    const Vector< Studentas > & vargsai,
    const Vector< Studentas > & kieti )
```

5.10.1.6 nuskaitytiFailaTestavimui()

```
void nuskaitytiFailaTestavimui (
    Vector< Studentas > & studentai,
    int kartai )
```

5.10.1.7 rikiuotiPagalGalutiniMed()

```
bool rikiuotiPagalGalutiniMed (
    const Studentas & A,
    const Studentas & B )
```

5.10.1.8 rikiuotiPagalGalutiniVid()

```
bool rikiuotiPagalGalutiniVid (
    const Studentas & A,
    const Studentas & B )
```

5.10.1.9 rikiuotiPavarde()

```
bool rikiuotiPavarde (
    const Studentas & A,
    const Studentas & B )
```

5.10.1.10 rikiuotiVarda()

```
bool rikiuotiVarda (
    const Studentas & A,
    const Studentas & B )
```

5.10.1.11 rodytiRez()

```
void rodytiRez (
    const Vector< Studentas > & studentai )
```

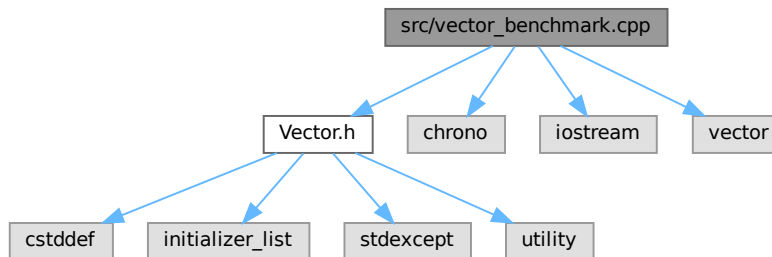
5.10.1.12 rodytiRezultatus()

```
void rodytiRezultatus (
    const Vector< Studentas > & studentai )
```

5.11 src/vector_benchmark.cpp Failo Nuoroda

```
#include "Vector.h"
#include <chrono>
#include <iostream>
#include <vector>
```

Įtraukimo priklausomybių diagrama vector_benchmark.cpp:



Funkcijos

- template<typename Konteineris >
double **matuotiPushBack** (int kiekis)
- template<typename Konteineris >
int **skaiciuotiPerskirstymus** (int kiekis)
- int **main** ()

5.11.1 Funkcijos Dokumentacija

5.11.1.1 main()

```
int main ( )
```

5.11.1.2 matuotiPushBack()

```
template<typename Konteineris >
double matuotiPushBack (
    int kiekis )
```

5.11.1.3 skaiciuotiPerskirstymus()

```
template<typename Konteineris >
int skaiciuotiPerskirstymus (
    int kiekis )
```

